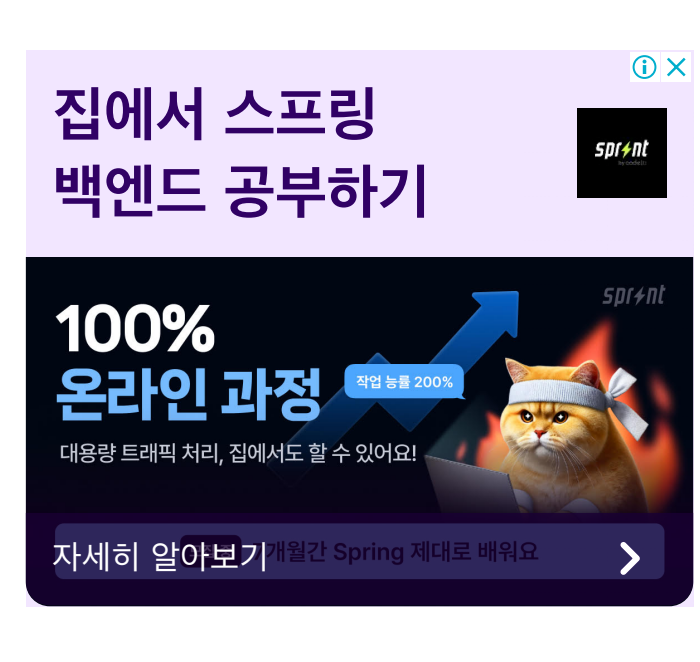
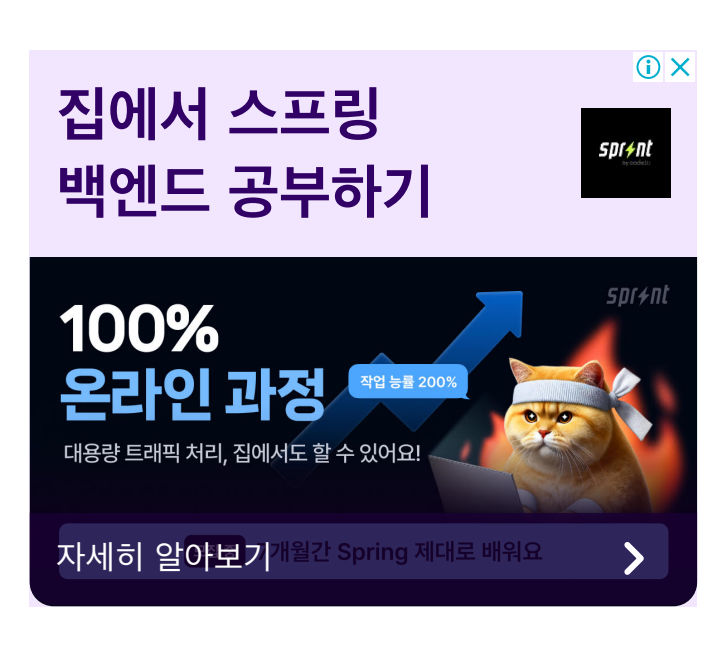
02-06 정수 인코딩(Integer Encoding)



집에서 스프링 백엔드 공부하기
100% 온라인 과정
대용량 트래픽 처리, 집에서도 할 수 있어요!
자세히 알아보기



집에서 스프링 백엔드 공부하기
100% 온라인 과정
대용량 트래픽 처리, 집에서도 할 수 있어요!
자세히 알아보기

컴퓨터는 텍스트보다는 숫자를 더 잘 처리 할 수 있습니다. 이를 위해 자연어 처리에서는 텍스트를 숫자로 바꾸는 여러가지 기법들이 있습니다. 그리고 그러한 기법들을 본격적으로 적용시키기 위한 첫 단계로 각 단어를 고유한 정수에 맵핑(mapping)시키는 전처리 작업이 필요할 때가 있습니다.

예를 들어 갖고 있는 텍스트에 단어가 5,000개가 있다면, 5,000개의 단어들 각각에 1번부터 5,000번까지 단어와 맵핑되는 고유한 정수. 다른 표현으로는 인덱스를 부여합니다. 가령, book은 150번, dog는 171번, love는 192번, books는 212번과 같이 숫자가 부여됩니다. 인덱스를 부여하는 방법은 여러 가지가 있을 수 있는데 랜덤으로 부여하기도 하지만, 보통은 단어 등장 빈도수를 기준으로 정렬한 뒤에 부여합니다.

1. 정수 인코딩(Integer Encoding)

왜 이러한 작업이 필요한 지에 대해서는 뒤에서 원-핫 인코딩 실습이나, 워드 임베딩 챕터 등에서 알아보기로 하고 여기서는 어떤 과정으로 단어에 정수 인덱스를 부여하는지에 대해서만 정리해보겠습니다.

단어에 정수를 부여하는 방법 중 하나로 단어를 빈도수 순으로 정렬한 단어 집합(vocabulary)을 만들고, 빈도수가 높은 순서대로 차례로 낮은 숫자부터 정수를 부여하는 방법이 있습니다. 이해를 돕기위해 단어의 빈도수가 적당하게 분포되도록 의도적으로 만든 텍스트 데이터를 가지고 실습해보겠습니다.

1) dictionary 사용하기

```
from nltk.tokenize import sent_tokenize
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
```

```
raw_text = "A barber is a person. a barber is good person. a barber is huge person. he Knew A Secre
t! The Secret He Kept is huge secret. Huge secret. His barber kept his word. a barber kept his wor
d. His barber kept his word. But keeping and keeping such a huge secret to himself was driving th
e barber crazy. the barber went up a huge mountain."
```

우선 여러 문장이 함께 있는 텍스트 데이터로부터 문장 토큰화를 수행해보겠습니다.

```
# 문장 토큰화
sentences = sent_tokenize(raw_text)
print(sentences)
```

```
['A barber is a person.', 'a barber is good person.', 'a barber is huge person.', 'he Knew A Secre
t!', 'The Secret He Kept is huge secret.', 'Huge secret.', 'His barber kept his word.', 'a barber k
ept his word.', 'His barber kept his word.', 'But keeping and keeping such a huge secret to himse
lf was driving the barber crazy.', 'the barber went up a huge mountain.']
```

기존의 텍스트 데이터가 문장 단위로 토큰화 된 것을 확인할 수 있습니다. 이제 정제 작업과 정규화 작업을 병행하며, 단어 토큰화를 수행합니다. 여기서는 단어들을 소문자화하여 단어의 개수를 통일시키고, 불용어와 단어 길이가 2이하인 경우에 대해서 단어를 일부 제외시켜주었습니다. 텍스트를 수치화하는 단계라는 것은 본격적으로 자연어 처리 작업에 들어간다는 의미이므로, 단어가 텍스트일 때만 할 수 있는 최대한의 전처리를 끝내놓아야 합니다.

HOW TO LOOT? IQ: 155 TRY

```
vocab = {}
preprocessed_sentences = []
stop_words = set(stopwords.words('english'))

for sentence in sentences:
    # 단어 토큰화
    tokenized_sentence = word_tokenize(sentence)
    result = []

    for word in tokenized_sentence:
        word = word.lower() # 모든 단어를 소문자화하여 단어의 개수를 줄인다.
        if word not in stop_words: # 단어 토큰화 된 결과에 대해서 불용어를 제거한다.
            if len(word) > 2: # 단어 길이가 2이하인 경우에 대하여 추가로 단어를 제거한다.
                result.append(word)
                if word not in vocab:
                    vocab[word] = 0
                vocab[word] += 1
    preprocessed_sentences.append(result)
print(preprocessed_sentences)
```

```
[['barber', 'person'], ['barber', 'good', 'person'], ['barber', 'huge', 'person'], ['knew', 'secre
t'], ['secret', 'kept', 'huge', 'secret'], ['huge', 'secret'], ['barber', 'kept', 'word'], ['barbe
r', 'kept', 'word'], ['barber', 'kept', 'secret'], ['keeping', 'keeping', 'huge', 'secret', 'drivin
g', 'barber', 'crazy'], ['barber', 'went', 'huge', 'mountain']]
```

현재 vocab에는 각 단어에 대한 빈도수가 기록되어져 있습니다. vocab을 출력해보겠습니다.

```
print('단어 집합 :',vocab)
```

```
단어 집합 : {'barber': 8, 'person': 3, 'good': 1, 'huge': 5, 'knew': 1, 'secret': 6, 'kept': 4, 'wor
d': 2, 'keeping': 2, 'driving': 1, 'crazy': 1, 'went': 1, 'mountain': 1}
```

파이썬의 딕셔너리 구조로 단어를 키(key)로, 단어에 대한 빈도수가 값(value)으로 저장되어져 있습니다. vocab에 단어를 입력하면 빈도수를 리턴합니다.

```
# 'barber'라는 단어의 빈도수 출력
print(vocab["barber"])
```

```
8
```

이제 빈도수가 높은 순서대로 정렬해보겠습니다.

HOW TO LOOT? IQ: 155 TRY

```
vocab_sorted = sorted(vocab.items(), key = lambda x:x[1], reverse = True)
print(vocab_sorted)
```

```
[('barber', 8), ('secret', 6), ('huge', 5), ('kept', 4), ('person', 3), ('word', 2), ('keeping',
2), ('good', 1), ('knew', 1), ('driving', 1), ('crazy', 1), ('went', 1), ('mountain', 1)]
```

높은 빈도수를 가진 단어일수록 낮은 정수를 부여합니다. 정수는 1부터 부여합니다.

```
word_to_index = {}
i = 0
for (word, frequency) in vocab_sorted :
    if frequency > 1 : # 빈도수가 작은 단어는 제외.
        i = i + 1
        word_to_index[word] = i

print(word_to_index)
```

```
{'barber': 1, 'secret': 2, 'huge': 3, 'kept': 4, 'person': 5, 'word': 6, 'keeping': 7}
```

1의 인덱스를 가진 단어가 가장 빈도수가 높은 단어가 됩니다. 그리고 이러한 작업을 수행하는 동시에 각 단어의 빈도수를 알 경우에만 할 수 있는 전처리인 빈도수가 적은 단어를 제외시키는 작업을 수행했습니다. 등장 빈도가 낮은 단어는 자연어 처리에서 의미를 가지지 않을 가능성이 높기 때문입니다. 여기서는 빈도수가 1인 단어들은 전부 제외시켰습니다.

자연어 처리를 하다보면, 텍스트 데이터에 있는 단어를 모두 사용하기 보다는 빈도수가 가장 높은 n개의 단어만 사용하고 싶은 경우가 많습니다. 위 단어들은 빈도수가 높은 순으로 낮은 정수가 부여되어져 있으므로 빈도수 상위 n개의 단어만 사용하고 싶다고하면 vocab에서 정수값이 1부터 n까지인 단어들만 사용하면 됩니다. 여기서는 상위 5개 단어만 사용한다고 가정하겠습니다.

```
vocab_size = 5

# 인덱스가 5 초과인 단어 제거
words_frequency = [word for word, index in word_to_index.items() if index >= vocab_size + 1]

# 해당 단어에 대한 인덱스 정보를 삭제
for w in words_frequency:
    del word_to_index[w]
print(word_to_index)
```

```
{'barber': 1, 'secret': 2, 'huge': 3, 'kept': 4, 'person': 5}
```

word_to_index에는 빈도수가 높은 상위 5개의 단어만 저장되었습니다. word_to_index를 사용하여 단어 토큰화가 된 상태로 저장된 sentences에 있는 각 단어를 정수로 바꾸는 작업을 하겠습니다.

예를 들어 sentences에서 첫번째 문장은 ['barber', 'person']이었는데, 이 문장에 대해서는 [1, 5]로 인코딩합니다. 그런데 두번째 문장인 ['barber', 'good', 'person']에는 더 이상 word_to_index에는 존재하지 않는 단어인 'good'이라는 단어가 있습니다.

이처럼 단어 집합에 존재하지 않는 단어들이 생기는 상황을 Out-Of-Vocabulary(단어 집합에 없는 단어) 문제라고 합니다. 약자로 'OOV 문제'라고도 합니다. word_to_index에 'OOV'란 단어를 새롭게 추가하고, 단어 집합에 없는 단어들은 'OOV'의 인덱스로 인코딩하겠습니다.

```
word_to_index['OOV'] = len(word_to_index) + 1
print(word_to_index)
```

```
{'barber': 1, 'secret': 2, 'huge': 3, 'kept': 4, 'person': 5, 'OOV': 6}
```

이제 word_to_index를 사용하여 sentences의 모든 단어들을 맵핑되는 정수로 인코딩하겠습니다.

```
encoded_sentences = []
for sentence in preprocessed_sentences:
    encoded_sentence = []
    for word in sentence:
        try:
            # 단어 집합에 있는 단어라면 해당 단어의 정수를 리턴.
            encoded_sentence.append(word_to_index[word])
        except KeyError:
            # 만약 단어 집합에 없는 단어라면 'OOV'의 정수를 리턴.
            encoded_sentence.append(word_to_index['OOV'])
    encoded_sentences.append(encoded_sentence)
print(encoded_sentences)
```

```
[[1, 5], [1, 6, 5], [1, 3, 5], [6, 2], [2, 4, 3, 2], [3, 2], [1, 4, 6], [1, 4, 6], [1, 4, 2], [6,
6, 3, 2, 6, 1, 6], [1, 6, 3, 6]]
```

지금까지 파이썬의 dictionary 자료형으로 정수 인코딩을 진행해보았습니다. 그런데 이보다는 좀 더 쉽게 하기 위해서 Counter, FreqDist, enumerate를 사용하거나, 케라스 토크나이저를 사용하는 것을 권장합니다.

2) Counter 사용하기

```
from collections import Counter
```

```
print(preprocessed_sentences)
```

```
[['barber', 'person'], ['barber', 'good', 'person'], ['barber', 'huge', 'person'], ['knew', 'secre
t'], ['secret', 'kept', 'huge', 'secret'], ['huge', 'secret'], ['barber', 'kept', 'word'], ['barbe
r', 'kept', 'word'], ['barber', 'kept', 'secret'], ['keeping', 'keeping', 'huge', 'secret', 'drivin
g', 'barber', 'crazy'], ['barber', 'went', 'huge', 'mountain']]
```

현재 sentences는 단어 토큰화가 된 결과가 저장되어져 있습니다. 단어 집합(vocabulary)을 만들기 위해서 sentences에서 문장의 경계인 [,]를 제거하고 단어들을 하나의 리스트로 만들겠습니다.

```
# words = np.hstack(preprocessed_sentences)으로도 수행 가능.
all_words_list = sum(preprocessed_sentences, [])
print(all_words_list)
```

```
['barber', 'person', 'barber', 'good', 'person', 'barber', 'huge', 'person', 'knew', 'secret', 'sec
ret', 'kept', 'huge', 'secret', 'huge', 'secret', 'barber', 'kept', 'word', 'barber', 'kept', 'wor
d', 'barber', 'kept', 'secret', 'keeping', 'keeping', 'huge', 'secret', 'driving', 'barber', 'craz
y', 'barber', 'went', 'huge', 'mountain']
```

이를 파이썬의 Counter()의 입력으로 사용하면 중복을 제거하고 단어의 빈도수를 기록합니다.

```
# 파이썬의 Counter 모듈을 이용하여 단어의 빈도수 카운트
vocab = Counter(all_words_list)
print(vocab)
```

```
Counter({'barber': 8, 'secret': 6, 'huge': 5, 'kept': 4, 'person': 3, 'word': 2, 'keeping': 2, 'goo
d': 1, 'knew': 1, 'driving': 1, 'crazy': 1, 'went': 1, 'mountain': 1})
```

단어를 키(key)로, 단어에 대한 빈도수가 값(value)으로 저장되어져 있습니다. vocab에 단어를 입력하면 빈도수를 리턴합니다.

```
print(vocab["barber"]) # 'barber'라는 단어의 빈도수 출력
```

```
8
```

barber란 단어가 총 8번 등장하였습니다. most_common()는 상위 빈도수를 가진 주어진 수의 단어만을 리턴합니다. 이를 사용하여 등장 빈도수가 높은 단어들을 원하는 개수만큼만 얻을 수 있습니다. 등장 빈도수 상위 5개의 단어만 단어 집합으로 저장해봅시다.

```
vocab_size = 5
vocab = vocab.most_common(vocab_size) # 등장 빈도수가 높은 상위 5개의 단어만 저장
vocab
```

```
[('barber', 8), ('secret', 6), ('huge', 5), ('kept', 4), ('person', 3)]
```

이제 높은 빈도수를 가진 단어일수록 낮은 정수 인덱스를 부여합니다.

```
word_to_index = {}
i = 0
for (word, frequency) in vocab :
    i = i + 1
    word_to_index[word] = i

print(word_to_index)
```

```
{'barber': 1, 'secret': 2, 'huge': 3, 'kept': 4, 'person': 5}
```

3) NLTK의 FreqDist 사용하기

NLTK에서는 빈도수 계산 도구인 FreqDist()를 지원합니다. 위에서 사용한 Counter()랑 같은 방법으로 사용할 수 있습니다.

```
from nltk import FreqDist
import numpy as np
```

```
# np.hstack으로 문장 구분을 제거
vocab = FreqDist(np.hstack(preprocessed_sentences))
```

단어를 키(key)로, 단어에 대한 빈도수가 값(value)으로 저장되어져 있습니다. vocab에 단어를 입력하면 빈도수를 리턴합니다.

```
print(vocab["barber"]) # 'barber'라는 단어의 빈도수 출력
```

```
8
```

barber란 단어가 총 8번 등장하였습니다. most_common()는 상위 빈도수를 가진 주어진 수의 단어만을 리턴합니다. 이를 사용하여 등장 빈도수가 높은 단어들을 원하는 개수만큼만 얻을 수 있습니다. 등장 빈도수 상위 5개의 단어만 단어 집합으로 저장해봅시다.

```
vocab_size = 5
vocab = vocab.most_common(vocab_size) # 등장 빈도수가 높은 상위 5개의 단어만 저장
print(vocab)
```

```
[('barber', 8), ('secret', 6), ('huge', 5), ('kept', 4), ('person', 3)]
```

앞서 Counter()를 사용했을 때와 결과가 같습니다. 이전 실습들과 마찬가지로 높은 빈도수를 가진 단어일수록 낮은 정수 인덱스를 부여합니다. 그런데 이번에는 enumerate()를 사용하여 좀 더 짧은 코드로 인덱스를 부여하겠습니다.

```
word_to_index = {word[0] : index + 1 for index, word in enumerate(vocab)}
print(word_to_index)
```

```
{'barber': 1, 'secret': 2, 'huge': 3, 'kept': 4, 'person': 5}
```

위와 같이 인덱스를 부여할 때는 enumerate()를 사용하는 것이 편리합니다. enumerate()에 대해서 간단히 소개해보겠습니다.

4) enumerate 이해하기

enumerate()는 순서가 있는 자료형(list, set, tuple, dictionary, string)을 입력으로 받아 인덱스를 순차적으로 함께 리턴한다는 특징이 있습니다. 간단한 예제를 통해 enumerate()를 이해해봅시다.

```
test_input = ['a', 'b', 'c', 'd', 'e']
for index, value in enumerate(test_input): # 입력의 순서대로 0부터 인덱스를 부여함.
  print("value : {}, index: {}".format(value, index))
```

```
value : a, index: 0
value : b, index: 1
value : c, index: 2
value : d, index: 3
value : e, index: 4
```

위의 출력 결과는 리스트의 모든 토큰에 대해서 인덱스가 순차적으로 증가되며 부여된 것을 보여줍니다.

---

2. 케라스(Keras)의 텍스트 전처리

케라스(Keras)는 기본적인 전처리를 위한 도구들을 제공합니다. 때로는 정수 인코딩을 위해서 케라스의 전처리 도구인 토크나이저를 사용하기도 하는데, 사용 방법과 그 특징에 대해서 이해해보겠습니다.

```
from tensorflow.keras.preprocessing.text  import Tokenizer
```

```
preprocessed_sentences = [['barber', 'person'], ['barber', 'good', 'person'], ['barber', 'huge', 'p
erson'], ['knew', 'secret'], ['secret', 'kept', 'huge', 'secret'], ['huge', 'secret'], ['barber', '
kept', 'word'], ['barber', 'kept', 'word'], ['barber', 'kept', 'secret'], ['keeping', 'keeping', 'h
uge', 'secret', 'driving', 'barber', 'crazy'], ['barber', 'went', 'huge', 'mountain']]
```

단어 토큰화까지 수행된 앞서 사용한 텍스트 데이터와 동일한 데이터를 사용합니다.

```
tokenizer = Tokenizer()

# fit_on_texts()안에 코퍼스를 입력으로 하면 빈도수를 기준으로 단어 집합을 생성.
tokenizer.fit_on_texts(preprocessed_sentences)
```

fit_on_texts는 입력한 텍스트로부터 단어 빈도수가 높은 순으로 낮은 정수 인덱스를 부여하는데, 정확히 앞서 설명한 정수 인코딩 작업이 이루어진다고 보면됩니다. 각 단어에 인덱스가 어떻게 부여되었는지를 보려면, word_index를 사용합니다.

```
print(tokenizer.word_index)
```

```
{'barber': 1, 'secret': 2, 'huge': 3, 'kept': 4, 'person': 5, 'word': 6, 'keeping': 7, 'good': 8, '
knew': 9, 'driving': 10, 'crazy': 11, 'went': 12, 'mountain': 13}
```

각 단어의 빈도수가 높은 순서대로 인덱스가 부여된 것을 확인할 수 있습니다. 각 단어가 카운트를 수행하였을 때 몇 개였는지를 보고자 한다면 word_counts를 사용합니다.

```
print(tokenizer.word_counts)
```

```
OrderedDict([('barber', 8), ('person', 3), ('good', 1), ('huge', 5), ('knew', 1), ('secret', 6), ('
kept', 4), ('word', 2), ('keeping', 2), ('driving', 1), ('crazy', 1), ('went', 1), ('mountain',
1)])
```

texts_to_sequences()는 입력으로 들어온 코퍼스에 대해서 각 단어를 이미 정해진 인덱스로 변환합니다.

```
print(tokenizer.texts_to_sequences(preprocessed_sentences))
```

```
[[1, 5], [1, 8, 5], [1, 3, 5], [9, 2], [2, 4, 3, 2], [3, 2], [1, 4, 6], [1, 4, 6], [1, 4, 2], [7,
7, 3, 2, 10, 1, 11], [1, 12, 3, 13]]
```

앞서 빈도수가 가장 높은 단어 n개만을 사용하기 위해서 most_common()을 사용했었습니다. 케라스 토크나이저에서는 tokenizer = Tokenizer(num_words=숫자)와 같은 방법으로 빈도수가 높은 상위 몇 개의 단어만 사용하겠다고 지정할 수 있습니다. 여기서는 1번 단어부터 5번 단어까지만 사용하겠습니다. 상위 5개 단어를 사용한다고 토크나이저를 재정의 해보겠습니다.

```
vocab_size = 5
tokenizer = Tokenizer(num_words = vocab_size + 1) # 상위 5개 단어만 사용
tokenizer.fit_on_texts(preprocessed_sentences)
```

num_words에서 +1을 더해서 값을 넣어주는 이유는 num_words는 숫자를 0부터 카운트합니다. 만약 5를 넣으면 0 ~ 4번 단어 보존을 의미하게 되므로 뒤의 실습에서 1번 단어부터 4번 단어만 남게됩니다. 그렇기 때문에 1 ~ 5번 단어까지 사용하고 싶다면 num_words에 숫자 5를 넣어주는 것이 아니라 5+1인 값을 넣어주어야 합니다.

실질적으로 숫자 0에 지정된 단어가 존재하지 않는데도 케라스 토크나이저가 숫자 0까지 단어 집합의 크기로 산정하는 이유는 자연어 처리에서 패딩(padding)이라는 작업 때문입니다. 이에 대해서는 뒤에 다루게 되므로 여기서는 케라스 토크나이저를 사용할 때는 숫자 0도 단어 집합의 크기로 고려해야한다고만 이해합시다.

다시 word_index를 확인해보겠습니다.

```
print(tokenizer.word_index)
```

```
{'barber': 1, 'secret': 2, 'huge': 3, 'kept': 4, 'person': 5, 'word': 6, 'keeping': 7, 'good': 8, '
knew': 9, 'driving': 10, 'crazy': 11, 'went': 12, 'mountain': 13}
```

상위 5개의 단어만 사용하겠다고 선언하였는데 여전히 13개의 단어가 모두 출력됩니다. word_counts를 확인해보겠습니다.

```
print(tokenizer.word_counts)
```

```
OrderedDict([('barber', 8), ('person', 3), ('good', 1), ('huge', 5), ('knew', 1), ('secret', 6), ('
kept', 4), ('word', 2), ('keeping', 2), ('driving', 1), ('crazy', 1), ('went', 1), ('mountain',
1)])
```

word_counts에서도 마찬가지로 13개의 단어가 모두 출력됩니다. 사실 실제 적용은 texts_to_sequences를 사용할 때 적용이 됩니다.

```
print(tokenizer.texts_to_sequences(preprocessed_sentences))
```

```
[[1, 5], [1, 5], [1, 3, 5], [2], [2, 4, 3, 2], [3, 2], [1, 4], [1, 4], [1, 4, 2], [3, 2, 1], [1,
3]]
```

코퍼스에 대해서 각 단어를 이미 정해진 인덱스로 변환하는데, 상위 5개의 단어만을 사용하겠다고 지정하였으므로 1번 단어부터 5번 단어까지만 보존되고 나머지 단어들은 제거된 것을 볼 수 있습니다. 경험상 굳이 필요하다고 생각하지는 않지만, 만약 word_index와 word_counts에서도 지정된 num_words만큼의 단어만 남기고 싶다면 아래의 코드도 방법입니다.

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(preprocessed_sentences)
```

```
vocab_size = 5
words_frequency = [word for word, index in tokenizer.word_index.items() if index >= vocab_size + 1]

# 인덱스가 5 초과인 단어 제거
for word in words_frequency:
    del tokenizer.word_index[word] # 해당 단어에 대한 인덱스 정보를 삭제
    del tokenizer.word_counts[word] # 해당 단어에 대한 카운트 정보를 삭제

print(tokenizer.word_index)
print(tokenizer.word_counts)
print(tokenizer.texts_to_sequences(preprocessed_sentences))
```

```
{'barber': 1, 'secret': 2, 'huge': 3, 'kept': 4, 'person': 5}
OrderedDict([('barber', 8), ('person', 3), ('huge', 5), ('secret', 6), ('kept', 4)])
[[1, 5], [1, 5], [1, 3, 5], [2], [2, 4, 3, 2], [3, 2], [1, 4], [1, 4], [1, 4, 2], [3, 2, 1], [1,
3]]
```

케라스 토크나이저는 기본적으로 단어 집합에 없는 단어인 OOV에 대해서는 단어를 정수로 바꾸는 과정에서 아예 단어를 제거한다는 특징이 있습니다. 단어 집합에 없는 단어들은 OOV로 간주하여 보존하고 싶다면 Tokenizer의 인자 oov_token을 사용합니다.

```
# 숫자 0과 OOV를 고려해서 단어 집합의 크기는 +2
vocab_size = 5
tokenizer = Tokenizer(num_words = vocab_size + 2, oov_token = 'OOV')
tokenizer.fit_on_texts(preprocessed_sentences)
```

만약 oov_token을 사용하기로 했다면 케라스 토크나이저는 기본적으로 'OOV'의 인덱스를 1로 합니다.

```
print('단어 OOV의 인덱스 : {}'.format(tokenizer.word_index['OOV']))
```

```
단어 OOV의 인덱스 : 1
```

이제 코퍼스에 대해서 정수 인코딩을 진행합니다.

```
print(tokenizer.texts_to_sequences(preprocessed_sentences))
```

```
[[2, 6], [2, 1, 6], [2, 4, 6], [1, 3], [3, 5, 4, 3], [4, 3], [2, 5, 1], [2, 5, 1], [2, 5, 3], [1,
1, 4, 3, 1, 2, 1], [2, 1, 4, 1]]
```

빈도수 상위 5개의 단어는 2 ~ 6까지의 인덱스를 가졌으며, 그 외 단어 집합에 없는 'good'과 같은 단어들은 전부 'OOV'의 인덱스인 1로 인코딩되었습니다.

마지막 편집일시 : 2022년 11월 14일 2:42 오후

댓글 5 피드백

이전글 : 02-05 정규 표현식(Regular Expression)
다음글 : 02-07 패딩(Padding)